Лабораторная работа 7. Тестирование нейронной сети на видеокадрах производственных объектов

Цель работы

Приобрести навыки проверки обученной нейронной сети на независимых изображениях и видеокадрах производственных объектов. Определить точность и полноту обнаружения, проанализировать ошибки и измерить продолжительность обработки кадра с учётом получения изображения, выполнения модели и сохранения результата.

Теоретические сведения

В лабораторной работе №6 выбрана и обучена модель для классов, размеченных в работе №5. В данной работе используется выбранный файл best.pt. Веса не изменяются. Последовательность предварительной обработки переносится из лабораторной работы №3 через функцию prepare_frame. Признаки объекта, рассмотренные в работе №4, используются при объяснении причин ошибок.

Проверка состоит из двух частей. Сначала по valid выбирается рабочий порог уверенности. Затем при зафиксированных параметрах оценивается test из лабораторной работы №5 и отдельно обрабатывается новый видеоролик. Для количественной оценки видео часть его кадров размечается вручную. Кадры нового тестового ролика и соседние кадры той же серии не должны входить в train или valid.

Результат обнаружения

Для каждого найденного объекта модель возвращает номер класса, координаты рамки и оценку уверенности. Уверенность используется для отбора предсказаний, но без отдельной калибровки её нельзя считать вероятностью безошибочного решения. Например, уверенность 0,9 не означает, что девять из десяти таких предсказаний обязательно будут верны.

Рабочий порог conf определяет, какие предсказания сохраняются. При его повышении часть ложных рамок может исчезнуть вместе с правильными обнаружениями слабозаметных объектов. Поэтому порог выбирают по размеченной valid с учётом точности и полноты. В примере используется максимум F1. При равенстве F1 выбирается большая точность, затем больший порог.

Для YOLOv8 параметр iou в predict управляет подавлением перекрывающихся предсказаний. Это отдельная операция, уменьшающая число близких рамок. Порог сопоставления с эталонной разметкой задаётся независимо. В работе используются iou = 0,7 для подавления и IoU ≥ 0,5 для зачёта обнаружения [1].

Сопоставление предсказаний с разметкой

Перекрытие двух рамок определяется отношением площади пересечения к площади объединения. На рисунке 1 приведён условный геометрический пример; он не является результатом выполнения модели.

$$IoU = \frac{S(Bп \cap Bэ)}{S(Bп \cup Bэ)}$$

Здесь Вп — предсказанная рамка, Вэ — эталонная рамка, S — площадь. Если рамки не пересекаются, IoU равен нулю; при полном совпадении — единице.

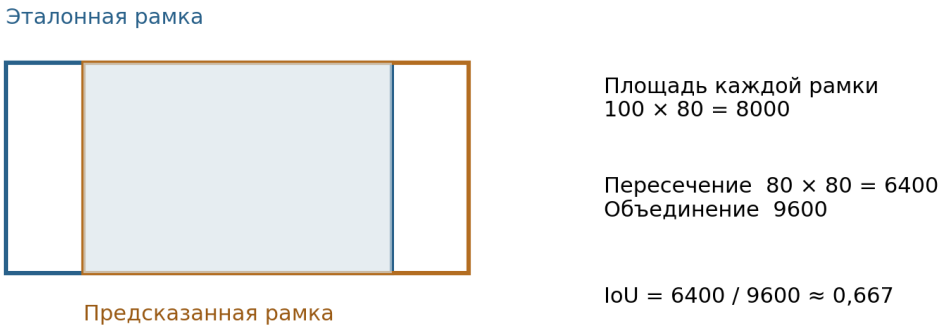


Рисунок 1 — Сопоставление предсказанной и эталонной рамок

Правильное обнаружение TP засчитывается при совпадении класса и достаточном IoU. Ложное обнаружение FP — предсказание, которому не соответствует подходящий эталон. Пропуск FN — эталонный объект, для которого не найдено правильного предсказания. Ошибка класса даёт FP для предсказанного класса и FN для эталонного. Повторная рамка одного объекта не увеличивает TP.

В учебном алгоритме предсказания упорядочиваются по убыванию уверенности. Каждое из них сопоставляется с ещё не использованной эталонной рамкой того же класса, имеющей наибольший IoU. Если перекрытие ниже порога, предсказание считается ложным. Одна эталонная рамка используется не более одного раза. Сопоставление выполняется отдельно на каждом кадре.

Точность и полнота

Точность P показывает долю правильных обнаружений среди всех предсказаний. Полнота R показывает долю найденных объектов среди всех размеченных. F1 объединяет эти показатели с одинаковым весом.

$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

$$F1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}$$

Если модель не выдала ни одной рамки, в учебной программе P принимается равной нулю. Если в оцениваемом наборе нет ни одного эталонного объекта класса, R и F1 для этого класса не определяются; программа возвращает None. Такой случай нельзя описывать как нулевое качество обнаружения отсутствующего класса.

Например, при TP = 2, FP = 1 и FN = 1 точность, полнота и F1 равны 2/3. Это расчётный пример. В отчёте используются значения, полученные на собственном наборе. Сначала суммируются TP, FP и FN по кадрам, затем рассчитываются общие показатели. Простое среднее покадровых значений может дать другой результат.

Показатели P, R и F1 относятся к выбранному рабочему порогу. Показатель mAP рассчитывается по зависимости точности от полноты при изменении порога и поэтому отдельно определяется штатной процедурой val [2]. Приведённый учебный алгоритм сопоставления предназначен для анализа ошибок при фиксированном пороге; он не воспроизводит весь расчёт mAP библиотеки.

Время обработки видеокадра

Для оценки быстродействия различаются время вызова predict и время полного цикла. Первый интервал включает подготовку входа внутри библиотеки, выполнение сети и обработку предсказаний. Второй дополнительно включает чтение и декодирование кадра, внешнюю предобработку, получение рамок на центральном процессоре, нанесение обозначений и запись результатов.

$$FPS = \frac{N}{T}$$

Здесь N — число измеренных кадров, T — суммарная продолжительность измеренных циклов в секундах. Кроме среднего времени фиксируются медиана и 95-й процентиль. Последний характеризует значение, которое не превышают приблизительно 95 % измерений. Начальная загрузка модели и предварительные прогревочные вызовы в измерение не включаются.

Для входа 25 кадров/с на один кадр приходится 40 мс. Если средняя продолжительность цикла больше, последовательная программа не успевает обрабатывать все поступающие кадры без накопления задержки. Даже при меньшем среднем времени отдельные задержки могут превышать этот интервал, поэтому дополнительно анализируется 95-й процентиль. Проверка файла без ограничения темпа поступления характеризует производительность программы, но не полную задержку реальной камеры.

Алгоритмы проверки и анализа

Алгоритм 1 сопоставляет рамки, подсчитывает TP, FP, FN и выявляет причины ошибок. Алгоритм 2 сравнивает результаты при заданных изменениях условий

наблюдения. Алгоритм 3 измеряет время выполнения и влияние режима обработки на быстродействие. Общие функции используются всеми студентами; дополнительное исследование определяется номером варианта.

Порядок выполнения работы

Сохраняются номер варианта, объект, классы и корневой каталог project предыдущих работ. Общая часть выполняется всеми студентами. Индивидуальное сравнение задаётся до просмотра тестовых результатов; по его итогам не подбирают новые параметры на тех же тестовых кадрах.

Общая часть

Шаг 1 Загрузка выбранной модели

Проверьте наличие best.pt, results/selected_model.json, dataset/data.yaml и preprocess.py из лабораторной работы №6. Сравните названия классов загруженной модели с data.yaml. Веса и imgsz берутся из описания выбранной модели. Устройство выполнения может отличаться от устройства обучения, но его характеристики необходимо указать в отчёте.

```
from pathlib import Path
import json
import torch
import yaml
from ultralytics import YOLO

selected = json.loads(Path("results/selected_model.json").read_text(
    encoding="utf-8"))
model = YOLO(selected["weights"])
size = int(selected["imgsz"])
device = 0 if torch.cuda.is_available() else "cpu"
cfg = yaml.safe_load(Path("dataset/data.yaml").read_text(
    encoding="utf-8"))
names = cfg["names"]
if isinstance(names, list):
    names = dict(enumerate(names))
else:
    names = {int(k): v for k, v in names.items()}
if model.names != names:
```

```
    raise ValueError("Порядок классов модели и набора различается")
print(model.names)
```

Шаг 2 Подготовка функций сопоставления

Создайте evaluate_frames.py. Функция evaluate принимает уже подготовленные и размеченные изображения. Повторный вызов prepare_frame для изображений dataset не выполняется. Все строки аннотаций должны содержать номер класса и четыре нормированных параметра рамки. Пустой проверенный файл соответствует кадру без объектов.

```
from pathlib import Path
import cv2

def iou(a, b):
    width = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
    height = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
    intersection = width * height
    area_a = max(0.0, a[2] - a[0]) * max(0.0, a[3] - a[1])
    area_b = max(0.0, b[2] - b[0]) * max(0.0, b[3] - b[1])
    union = area_a + area_b - intersection
    return intersection / union if union > 0 else 0.0

def match_counts(truth, predicted, match_iou=0.5):
    used = set()
    tp = 0
    for cls, confidence, box in sorted(
            predicted, key=lambda item: item[1], reverse=True):
        candidates = [(iou(box, other), index)
                      for index, (label, other) in enumerate(truth)
                      if index not in used and label == cls]
        overlap, index = max(candidates, default=(0.0, -1))
        if index >= 0 and overlap >= match_iou:
            used.add(index)
            tp += 1
    return tp, len(predicted) - tp, len(truth) - tp

def scores(tp, fp, fn):
```

```
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else None
    f1 = 2 * tp / (2 * tp + fp + fn) if tp + fn else None
    return dict(TP=tp, FP=fp, FN=fn,
                P=precision, R=recall, F1=f1)


def read_truth(path, width, height, n_classes):
    truth = []
    for line in path.read_text(encoding="utf-8").splitlines():
        if not line.strip():
            continue
        parts = line.split()
        if len(parts) != 5:
            raise ValueError(f"Нужна рамочная разметка: {path}")
        cls = int(parts[0])
        xc, yc, bw, bh = map(float, parts[1:])
        if not 0 <= cls < n_classes or min(bw, bh) <= 0:
            raise ValueError(f"Некорректная разметка: {path}")
        box = [(xc - bw / 2) * width, (yc - bh / 2) * height,
               (xc + bw / 2) * width, (yc + bh / 2) * height]
        truth.append((cls, box))
    return truth


def evaluate(model, image_dir, label_dir, conf, size,
             device="cpu", match_iou=0.5, transform=None):
    extensions = {".jpg", ".jpeg", ".png", ".bmp"}
    files = [p for p in sorted(Path(image_dir).iterdir())
             if p.suffix.lower() in extensions]
    if not files:
        raise ValueError("Нет изображений для оценки")
    counts = {cls: [0, 0, 0] for cls in model.names}
    for path in files:
        image = cv2.imread(str(path))
        if image is None:
            raise ValueError(f"Не читается изображение: {path}")
```

```
        height, width = image.shape[:2]
        label = Path(label_dir) / (path.stem + ".txt")
        truth = read_truth(label, width, height, len(model.names))
        if transform is not None:
            image = transform(image)
            if image.shape[:2] != (height, width):
                raise ValueError("Преобразование изменило геометрию")
        result = model.predict(
            image, imgsz=size, conf=conf, iou=0.7,
            device=device, rect=False, verbose=False)[0]
        boxes = result.boxes.cpu()
        predicted = [(int(cls), float(score), box)
                     for cls, score, box in zip(
                         boxes.cls.tolist(), boxes.conf.tolist(),
                         boxes.xyxy.tolist())]
        for cls in counts:
            gt = [(c, b) for c, b in truth if c == cls]
            pr = [(c, s, b) for c, s, b in predicted if c == cls]
            values = match_counts(gt, pr, match_iou)
            counts[cls] = [a + b for a, b in zip(counts[cls], values)]
    total = [sum(values[i] for values in counts.values())
             for i in range(3)]
    per_class = {model.names[cls]: scores(*values)
                 for cls, values in counts.items()}
    return dict(images=len(files), **scores(*total), classes=per_class)
```

Перед применением к своему набору проверьте match_counts на трёх простых случаях: одна правильная рамка; две одинаковые рамки одного объекта; рамка с неверным классом. Ожидаемые значения соответственно (1, 0, 0), (1, 1, 0) и (0, 1, 1). Здесь порядок чисел — TP, FP, FN. Такая проверка позволяет обнаружить повторный зачёт одной эталонной рамки.

Шаг 3 Выбор рабочего порога на valid

Создайте select_threshold.py. Программа проверяет три заранее заданных порога, использует одну и ту же модель и сохраняет выбранное значение. Если в valid нет эталонных объектов, подбор порога прекращается. Веса при выполнении программы не изменяются.

```
from pathlib import Path
import json
import torch
from ultralytics import YOLO
from evaluate_frames import evaluate

cfg = json.loads(Path("results/selected_model.json").read_text(
    encoding="utf-8"))
model = YOLO(cfg["weights"])
device = 0 if torch.cuda.is_available() else "cpu"
rows = []
for conf in (0.15, 0.35, 0.55):
    result = evaluate(model, "dataset/valid/images",
                      "dataset/valid/labels", conf,
                      cfg["imgsz"], device)
    if result["F1"] is None:
        raise ValueError("В valid нет размеченных объектов")
    rows.append(dict(conf=conf, **result))
best = max(rows, key=lambda row: (row["F1"], row["P"], row["conf"]))
cfg.update(conf=best["conf"], nms_iou=0.7, match_iou=0.5)
Path("results/lr7_mode.json").write_text(
    json.dumps(cfg, ensure_ascii=False, indent=2), encoding="utf-8")
Path("results/lr7_thresholds.json").write_text(
    json.dumps(rows, ensure_ascii=False, indent=2), encoding="utf-8")
print("Выбранный порог:", best["conf"])
python select_threshold.py
```

Составьте таблицу conf, TP, FP, FN, P, R и F1 на valid. Изменения порога исследуются только здесь. После выбора сохраните lr7_mode.json и до завершения итоговой проверки не меняйте веса, размер входа, предобработку и рабочий порог. Для индивидуального задания заранее запишите дополнительное сравнение из таблицы вариантов.

Шаг 4 Итоговая оценка на test

Загрузите режим из lr7_mode.json и выполните следующий фрагмент. Он записывает P, R и F1 при рабочем пороге. Дополнительно штатная процедура рассчитывает mAP на test

из лабораторной работы №5. Низкий порог conf в вызове val нужен для расчёта mAP и не заменяет рабочее значение из lr7_mode.json.

```
from pathlib import Path
import json
import torch
from ultralytics import YOLO
from evaluate_frames import evaluate

cfg = json.loads(Path("results/lr7_mode.json").read_text(
   encoding="utf-8"))
device = 0 if torch.cuda.is_available() else "cpu"
model = YOLO(cfg["weights"])
result = evaluate(model, "dataset/test/images",
                  "dataset/test/labels", cfg["conf"],
                  cfg["imgsz"], device, cfg["match_iou"])
metrics = model.val(
   data="dataset/data.yaml", split="test", imgsz=cfg["imgsz"],
   device=device, batch=1, workers=0, conf=0.001,
   iou=cfg["nms_iou"], plots=True,
   project="runs/lr7", name="test_final")
result.update(map50=float(metrics.box.map50),
              map50_95=float(metrics.box.map))
Path("results/lr7_test.json").write_text(
   json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
print(result)
```

Файл lr7_test.json содержит итоговую оценку исходного test, а не valid. Если проверка выявила необходимость изменения модели, этот результат сначала фиксируется как итог текущего опыта. Последующее обучение относится к новому опыту и требует отдельной независимой итоговой проверки.

Шаг 5 Подготовка нового видеоролика

Запишите новый ролик продолжительностью не менее 20–30 секунд с теми же классами объектов. Включите изменение положения, масштаба или освещения и, если возможно, участок без объектов. Сохраните ролик как data/video/test_video.mp4. Для примера ниже предполагается постоянная частота кадров, известная из файла. При использовании видео с переменной частотой необходимо применять его временные метки.

Для количественной оценки выберите не менее 30 различающихся кадров из разных участков нового ролика. Извлекайте их с исходным номером в имени, например test_video_000125.jpg. Перед разметкой примените prepare_frame ровно один раз. Разметьте кадры по тем же правилам и в том же порядке классов, что и в лабораторной работе №5. Кадры без объектов также включаются в оценку и получают пустые файлы .txt.

Разместите изображения в video_test/images, а аннотации — в video_test/labels. В этом каталоге хранятся подготовленные кадры, поэтому evaluate не вызывает prepare_frame. Результаты на test и video_test приводятся раздельно. При малом числе объектов обязательно укажите число эталонных рамок каждого класса: одинаковые значения F1 на десяти и на тысяче объектов имеют разную убедительность.

Шаг 6 Обработка видео и измерение времени

Создайте run_video.py. Программа читает кадры последовательно, выполняет предобработку, вызывает модель, сохраняет видео с рамками и два журнала. Файл detections.csv содержит строки обнаружений, а timings.csv — запись для каждого кадра, в том числе для кадра без объектов. Координаты относятся к подготовленному кадру, на котором изображаются рамки [1, 3].

```
from pathlib import Path
import csv
import json
import time
import cv2
import numpy as np
import torch
from ultralytics import YOLO
from preprocess import prepare_frame

SOURCE = "data/video/test_video.mp4"
OUTPUT = Path("results/lr7_video")
SAVE_VIDEO = True
cfg = json.loads(Path("results/lr7_mode.json").read_text(
   encoding="utf-8"))
device = 0 if torch.cuda.is_available() else "cpu"
model = YOLO(cfg["weights"])

def sync():
```

```
    if device != "cpu":
        torch.cuda.synchronize(device)

def predict(frame):
    return model.predict(
        frame, imgsz=cfg["imgsz"], conf=cfg["conf"],
        iou=cfg["nms_iou"], device=device,
        rect=False, verbose=False)[0]

OUTPUT.mkdir(parents=True, exist_ok=False)
cap = cv2.VideoCapture(SOURCE)
fps = cap.get(cv2.CAP_PROP_FPS)
ok, raw = cap.read()
if not ok or not np.isfinite(fps) or fps <= 0:
    cap.release()
    raise ValueError("Проверьте видео и его частоту кадров")
prepared = prepare_frame(raw)
height, width = prepared.shape[:2]
for _ in range(5):
    predict(prepared)
sync()
cap.release()
cap = cv2.VideoCapture(SOURCE)  # начать заново после прогрева
writer = None
if SAVE_VIDEO:
    writer = cv2.VideoWriter(
        str(OUTPUT / "annotated.mp4"),
        cv2.VideoWriter_fourcc(*"mp4v"), fps, (width, height))
    if not writer.isOpened():
        cap.release()
        raise RuntimeError("Не удалось открыть запись видео")
times = []
predict_times = []
index = 0
try:
```

```
with (OUTPUT / "detections.csv").open(
        "w", newline="", encoding="utf-8-sig") as df, \
     (OUTPUT / "timings.csv").open(
        "w", newline="", encoding="utf-8-sig") as tf:
    detections = csv.writer(df)
    timings = csv.writer(tf)
    detections.writerow(["frame", "time_s", "class", "conf",
                         "x1", "y1", "x2", "y2"])
    timings.writerow(["frame", "time_s", "predict_ms",
                      "cycle_ms", "objects"])
    while True:
        sync()
        started = time.perf_counter()
        ok, raw = cap.read()
        if not ok:
            break
        frame = prepare_frame(raw)
        if frame.shape[:2] != (height, width):
            raise ValueError("Размер подготовленного кадра изменился")
        sync()
        before_predict = time.perf_counter()
        result = predict(frame)
        sync()
        predict_s = time.perf_counter() - before_predict
        boxes = result.boxes.cpu()
        annotated = result.plot()
        if writer is not None:
            writer.write(annotated)
        for cls, score, box in zip(
                boxes.cls.tolist(), boxes.conf.tolist(),
                boxes.xyxy.tolist()):
            detections.writerow([index, index / fps,
                                 model.names[int(cls)], score, *box])
        cycle_s = time.perf_counter() - started
        times.append(cycle_s)
```

```
            predict_times.append(predict_s)
            timings.writerow([index, index / fps, predict_s * 1000,
                              cycle_s * 1000, len(boxes)])
            index += 1
    finally:
        cap.release()
        if writer is not None:
            writer.release()
    if not times:
        raise RuntimeError("Нет обработанных кадров")
    budget = 1.0 / fps
    summary = dict(
        frames=index, source_fps=fps, imgsz=cfg["imgsz"],
        conf=cfg["conf"], device=str(device), save_video=SAVE_VIDEO,
        predict_mean_ms=float(np.mean(predict_times) * 1000),
        cycle_mean_ms=float(np.mean(times) * 1000),
        cycle_median_ms=float(np.median(times) * 1000),
        cycle_p95_ms=float(np.percentile(times, 95) * 1000),
        processing_fps=len(times) / sum(times),
        over_budget_percent=100 * sum(t > budget for t in times) / len(times),
    )
    (OUTPUT / "summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    print(summary)
    python run_video.py
```

Время cycle_ms измеряется до записи собственной строки timings.csv. Оно включает нанесение рамок даже при SAVE_VIDEO = False, запись строк обнаружений и, при включённом сохранении, вызов VideoWriter.write. Закрытие файла, завершающая работа кодировщика и принудительный сброс данных на носитель в этот интервал не входят. При выводе о работе производственной системы границы измерения необходимо сохранять явно.

Программа не пропускает кадры намеренно и записывает видео с частотой исходного файла. Поэтому плавное воспроизведение annotated.mp4 не доказывает, что обработка выполнялась с этой частотой. Скорость оценивается по журналу и processing_fps. Для нового запуска задайте другой OUTPUT, чтобы сохранить результаты предыдущего.

Если prepare_frame изменяет размер или добавляет поля, координаты CSV и подготовленного изображения относятся к преобразованному кадру. Для пересчёта в исходный кадр необходимо учесть масштаб и ширину полей. Прямое сравнение таких координат с аннотациями исходного разрешения недопустимо.

Шаг 7 Количественная оценка видеокадров

Оцените video_test функцией evaluate при зафиксированном режиме. Сохраните отдельный результат. Для изображения без предсказаний, на котором есть эталонный объект, алгоритм должен записать пропуск. Отсутствие строки в detections.csv не означает, что кадр не обрабатывался: его наличие проверяется по timings.csv.

```
video_result = evaluate(
    model, "video_test/images", "video_test/labels",
    cfg["conf"], cfg["imgsz"], device, cfg["match_iou"])
Path("results/lr7_video_quality.json").write_text(
    json.dumps(video_result, ensure_ascii=False, indent=2),
    encoding="utf-8")
print(video_result)
```

Фрагмент выполняется после загрузки cfg, model и device по образцу шага 4, с теми же импортами. Приведите общую таблицу и таблицу по классам. Число эталонных объектов равно TP + FN. Укажите число размеченных кадров; не называйте этот объём числом независимых производственных ситуаций, если несколько кадров получены из одного участка ролика.

Набор	**Кадры**	**Эталонных объектов**	**TP**	**FP**	**FN**	**P**	**R**	**F1**
test из ЛР 5	—	—	—	—	—	—	—	—
Новый ролик video_test	—	—	—	—	—	—	—	—

Индивидуальная часть

Шаг 8 Выполнение дополнительного исследования

Для вариантов 1–4 дополните анализ ошибок по таблице заданий. Используйте match_counts, результаты предсказаний и эталонные рамки. Найдите и покажите кадры, объясняющие основные ошибки. Для оценки малых объектов площадь вычисляется по эталонной рамке относительно площади подготовленного кадра; изменять её по предсказанию нельзя.

Для вариантов 5–7 сравните условия наблюдения. При изменении яркости и размытии геометрия кадра сохраняется, поэтому используются прежние аннотации. Условия создаются после подготовки кадра, без дополнительного изменения размера. Такие изображения являются преобразованиями одних и тех же тестовых кадров и не считаются новым независимым набором.

```
import cv2
import numpy as np

def brightness(image, factor):
    value = image.astype(np.float32) * factor
    return np.clip(value, 0, 255).astype(np.uint8)

# Вариант 5: factor принимает 0.6, 1.0 и 1.4.
dark_result = evaluate(
    model, "video_test/images", "video_test/labels",
    cfg["conf"], cfg["imgsz"], device,
    transform=lambda image: brightness(image, 0.6))

# Вариант 6: без размытия, затем ядра 3 x 3 и 7 x 7.
blur_result = evaluate(
    model, "video_test/images", "video_test/labels",
    cfg["conf"], cfg["imgsz"], device,
    transform=lambda image: cv2.GaussianBlur(image, (7, 7), 0))
```

Для варианта 7 вручную выделите две группы минимум по 10 кадров: объект виден полностью и объект частично перекрыт. Правила разметки сохраняются из лабораторной работы №5. Запишите признак группы отдельно, оцените обе группы и укажите их фактический состав. Различие показателей нельзя объяснять только перекрытием, если группы одновременно различаются освещением или размером объектов.

Для вариантов 8–10 используйте timings.csv и дополнительные заранее заданные режимы. Выполните по три измерительных прохода одного и того же ролика после прогрева. Сохраняйте разрешение, устройство, порог и состав кадров; меняется только фактор, указанный в таблице. Результаты по повторениям приведите отдельно, затем сравните медианы итоговых показателей.

В варианте 8 сравнение размеров входа задаётся до теста и характеризует компромисс между временем и качеством. Оно не заменяет режим, выбранный на valid. Для

каждого размера дополнительно определите P, R и F1 на одинаковых кадрах video_test. В варианте 9 нанесение рамок и запись CSV выполняются в обоих режимах; включение SAVE_VIDEO изменяет только запись видео.

Шаг 9 Анализ ошибок и вывод

Сохраните не менее пяти кадров с пояснениями, в том числе с правильным обнаружением и имеющимися ошибками. Разделите причины по наблюдаемым признакам: малая проекция объекта, перекрытие, блик, смазывание, сходный фон, недостаток примеров класса или неточность разметки. Не приписывайте причину только по одному значению уверенности.

В выводе укажите качество на независимых данных, наиболее трудный класс, условия появления ошибок, среднее время и 95-й процентиль полного цикла. Сопоставьте время с интервалом поступления кадров исходного видео. Сформулируйте, какие изменения данных или обработки следует проверить в новом опыте. Высокие показатели на небольшом фрагменте видео не следует распространять на все условия производства.

Работа считается выполненной, если модель проверена на test и размеченных кадрах нового ролика, рабочий порог выбран по valid, сохранены журналы и видео, получены измерения времени, выполнено исследование по варианту и объяснены обнаруженные ошибки. Отрицательный результат отдельного опыта допустим при корректном проведении и анализе.

Содержание отчёта

1. Цель работы, номер варианта, объект и перечень классов.

2. Сведения о модели из лабораторной работы №6 и выбранной версии набора.

3. Таблица подбора рабочего порога на valid и зафиксированный режим.

4. Источник test и нового ролика, параметры видео, число размеченных кадров и объектов по классам.

5. Описание согласования предобработки и координат.

6. Листинг сопоставления рамок, результаты проверки TP, FP и FN на простых случаях.

7. Таблицы P, R, F1 для test и video_test; значения mAP для test.

8. Не менее пяти кадров с разбором правильных и ошибочных предсказаний.

9. Таблица времени predict и полного цикла, медиана, 95-й процентиль и фактическая производительность.

10. Листинг и результаты дополнительного исследования по варианту.

11. Файлы annotated.mp4, detections.csv, timings.csv и описание режима.

12. Вывод о пригодности модели в исследованных условиях и необходимых следующих опытах.

Варианты заданий

Объекты и классы сохраняются из лабораторных работ №5 и №6. Все студенты выполняют общую часть; индивидуальная часть различается видом анализа. Значения, указанные в заданиях, задают условия учебного опыта и не являются нормативами приёмки производственной системы.

№	Производственный объект	Индивидуальное задание
1	Детали на конвейере	Алгоритм 1 — определить TP, FP, FN, P, R, F1 отдельно для gear, washer, nut; показать наиболее частую ошибку
2	Печатная плата	Алгоритм 1 — разделить эталонные объекты по доле площади рамки менее 1 % кадра и не менее 1 %; сравнить полноту групп
3	Упаковка и тара	Алгоритм 1 — оценить ложные рамки на не менее 10 размеченных кадрах без объектов; привести FP на кадр и долю кадров с FP
4	Маркировка	Алгоритм 1 — сравнить P, R, F1 при порогах сопоставления IoU 0,5 и 0,75; веса и conf фиксированы
5	Металлическая поверхность	Алгоритм 2 — сравнить множители яркости 0,6; 1,0; 1,4 на одинаковых кадрах; оценить изменение полноты дефектов
6	Пищевой продукт	Алгоритм 2 — сравнить исходные кадры и гауссово размытие с ядрами 3 × 3 и 7 × 7; построить таблицу по классам
7	Сварной шов	Алгоритм 2 — сравнить полностью видимые и частично перекрытые объекты в двух размеченных группах минимум по 10 кадров

№	Производственный объект	Индивидуальное задание
8	Текстильное полотно	Алгоритм 3 — сравнить imgsz = 416 и 640 по P, R, F1 и времени полного цикла; выполнить три прохода для каждого размера
9	Инструмент на складе	Алгоритм 3 — сравнить SAVE_VIDEO = True и False при одинаковом нанесении рамок и записи CSV; три прохода каждого режима
10	Автомобильная деталь	Алгоритм 3 — определить медиану, 95-й процентиль и долю циклов более 40 мс; три прохода, сравнение с режимом 25 кадров/с

Для варианта 2 сначала сопоставляются все рамки обычным способом, затем каждый эталонный объект относится к своей группе и учитывается как найденный или пропущенный. Предсказания не нужно отдельно сопоставлять с каждой группой: это может повторно зачесть одну рамку. Если группа не содержит объектов, её полнота не рассчитывается.

Контрольные вопросы

1. Какие параметры модели необходимо зафиксировать до итогового тестирования?
2. Почему рабочий порог уверенности выбирается по valid, а не по test?
3. Чем уверенность модели отличается от измеренной точности обнаружения?
4. Что показывает IoU и почему совпадения класса недостаточно для зачёта рамки?
5. Как учитываются повторная рамка и ошибка класса?
6. Почему одна эталонная рамка не должна увеличивать TP несколько раз?
7. Чем различаются P, R, F1 и mAP?
8. Почему по видео без эталонной разметки нельзя вычислить полноту обнаружения?
9. Для чего журнал времени должен содержать записи кадров без объектов?
10. Какие операции включены в измерение полного цикла и какие выполняются вне него?
11. Почему частота сохранённого видео не доказывает обработку в реальном времени?
12. Зачем перед измерением выполняются прогрев модели и синхронизация графического ускорителя?
13. Что показывает 95-й процентиль времени обработки?

14. Какие ограничения имеет ваш индивидуальный опыт и какие дополнительные данные нужны для проверки вывода?